SOFTWARE REQUIREMENTS SPECIFICATION

Campus Skill Hub

Student Skill Exchange Platform

A Peer-to-Peer Learning & Skill Exchange Web Platform for College Students

Document Type	Software Requirements Specification (SRS)
IEEE Standard Reference	IEEE 830 / ISO-IEC-IEEE 29148 (Adapted)
Document Version	1.0
Prepared For	Front End Engineering
Status	Draft for Review

Prepared by: Prem Miglani (NC)

Department of Computer Science / Information Technology

Academic Year 2026

Document Control

Revision History

Version	Date	Description	Author
0.1	Draft	Initial requirement gathering and scope definition	Prem Miglani
1.0	Current	Complete SRS aligned with IEEE 830 structure	Prem Miglani

Approval

Role	Name	Signature	Date
Project Guide / Faculty Mentor			
Team Lead / Student Coordinator			

Table of Contents

1. Introduction

1. Purpose
2. Document Conventions
3. Intended Audience and Reading Suggestions
4. Project Scope
5. Definitions, Acronyms and Abbreviations
6. References

2. Overall Description

1. Product Perspective
2. Product Features (Summary)
3. User Classes and Characteristics
4. Operating Environment
5. Design and Implementation Constraints
6. Assumptions and Dependencies

3. System Features (Functional Requirements)

4. Data Requirements / Conceptual Data Model

5. External Interface Requirements

1. User Interfaces
2. Hardware Interfaces
3. Software Interfaces
4. Communication Interfaces

6. Non-Functional Requirements

7. System Architecture Overview

8. Use Case Diagrams and Descriptions

9. Future Scope / Planned API and Database Integration

10. Testing Requirements

11. Other Requirements

Appendix A: Glossary

Appendix B: Analysis Models

1. Introduction

1. Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for **Campus Skill Hub**, a frontend web platform designed to connect students within a college community who wish to learn new skills with peers who are willing to teach them. This document is intended to serve as the primary reference for the design, development, testing, and evaluation of the system. It follows the general structure recommended by the IEEE 830 / ISO-IEC-IEEE 29148 standards for Software Requirements Specifications, adapted to a realistic scope suitable for a college-level software engineering project.

2. Document Conventions

Requirements in this document are uniquely identified using the format **FR-XX** for Functional Requirements and **NFR-XX** for Non-Functional Requirements, where XX is a sequential number. Priority levels (High / Medium / Low) are assigned to each functional requirement to guide development sequencing. Tables are used extensively to present requirements, data fields, and interface details in a structured, evaluator-friendly format.

3. Intended Audience and Reading Suggestions

Audience	Purpose of Reading this Document
Faculty Guide / Evaluator	To assess project scope, feasibility, and completeness against academic evaluation criteria.
Student Development Team	To use as the primary blueprint for design, coding, and testing.
UI/UX Designer (team member)	To understand screen-level requirements and user flows for wireframing/prototyping.
Future Backend/Database Developer	To plan API contracts and database schema for later phases.
Software Tester / QA	To derive test cases directly from functional and non-functional requirements.

It is recommended to read Section 2 (Overall Description) first for context, followed by Section 3 (System Features) for detailed functional requirements, and Section 6 for quality attributes.

1.4 Project Scope

Campus Skill Hub is a **frontend-focused web application** (with a simulated/mock data layer in the current phase, and a clearly defined path to real backend/API/database integration in a future phase — see Section 9). The system enables students to create profiles, list skills they can teach, browse or search for skills they want to learn, connect with matching peers, and manage simple skill-exchange requests/sessions. The scope is deliberately kept realistic for a small student team (3–5 members) to design, build, and demo within a single academic semester.

In Scope

- Student registration and login (mock authentication in current phase).
- Creation and management of a personal profile listing skills to teach and skills to learn.

- Searching and filtering peers/skills by category, skill name, and availability.
- Sending, accepting, and declining skill-exchange / session requests.
- A simple in-app messaging/notification indicator for request updates.
- A ratings and feedback mechanism after a completed skill session.
- An admin-style moderation view for managing users and reported content.
- Responsive UI usable on desktop and mobile browsers.

Out of Scope (Current Phase)

- Real-time video/audio calling for skill sessions (external tools like Google Meet may be linked instead).
- Payment processing or monetization of skill exchange.
- Native mobile applications (Android/iOS) — web-responsive only.
- Fully persistent production-grade backend and database (planned for future phase, Section 9).
- AI-based skill/peer recommendation engine (identified as a future enhancement).

1.5 Definitions, Acronyms and Abbreviations

Term	Definition
SRS	Software Requirements Specification
Skill Provider	A student user offering to teach a particular skill
Skill Seeker	A student user looking to learn a particular skill
Skill Exchange Request	A request sent from a seeker to a provider to initiate a learning session
Session	A scheduled peer-to-peer learning interaction between a seeker and a provider
Admin	A user role responsible for platform moderation and oversight
UI/UX	User Interface / User Experience
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
Mock Data Layer	Simulated data (e.g., JSON/local storage) used in place of a live backend during the current phase

1.6 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 29148:2018, Systems and Software Engineering — Requirements Engineering.
- Nielsen Norman Group — Usability Heuristics for User Interface Design.
- W3C Web Content Accessibility Guidelines (WCAG) 2.1.

2. Overall Description

1. Product Perspective

Campus Skill Hub is a new, standalone, self-contained web platform. It is not a modification of an existing system. In its current phase, it is implemented as a frontend-driven application that uses a mock/local data layer to simulate persistence, allowing the full user experience (registration, matching, requests, ratings) to be demonstrated without a live production backend. The architecture is designed so that the mock data layer can be replaced with real REST APIs and a database in a later phase (see Section 9) without major redesign of the UI layer.

2. Product Features (Summary)

Feature Area	Summary
User Onboarding	Registration, login, and profile setup with academic and skill details.
Skill Listings	Users list skills they can teach and skills they want to learn, with proficiency levels.
Discovery & Search	Browse and filter peers/skills by category, keyword, department, and availability.
Matching & Requests	Send/accept/decline skill-exchange requests between seekers and providers.
Session Management	Track upcoming, completed, and cancelled skill-exchange sessions.
Feedback & Ratings	Rate and review peers after a completed session to build trust.
Notifications	In-app indicators for new requests, acceptances, and messages.
Admin Moderation	Admin dashboard to manage users, categories, and reported content.

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise
Student (Skill Seeker)	Browses/searches for skills, sends exchange requests, rates sessions.	Basic web/browser familiarity
Student (Skill Provider)	Lists teachable skills, accepts/declines requests, conducts sessions.	Basic web/browser familiarity
Hybrid User	Most students act as both seeker and provider using the same account.	Basic web/browser familiarity
Administrator	Moderates users/content, manages skill categories, views platform analytics.	Moderate; comfortable with admin dashboards

4. Operating Environment

- Client: Modern web browsers — Google Chrome, Mozilla Firefox, Microsoft Edge, Safari (latest two versions).
- Device support: Desktop, laptop, tablet, and mobile browsers via responsive design.

- Current phase data layer: Browser local storage / static JSON mock data (no server required to run the demo).
- Future phase (Section 9): Node.js/Express or similar backend with a relational or NoSQL database, deployed on a cloud platform.

2.5 Design and Implementation Constraints

- The project must be completed by a small student team within one academic semester; scope is intentionally limited.
- Current phase must function without requiring a live backend server, for ease of evaluation and demonstration.
- UI must follow the defined navy blue / royal blue / light blue color theme for visual consistency (see Section 5.1).
- The system must be built using standard, well-documented frontend technologies (e.g., HTML5, CSS3, JavaScript / React) to keep the codebase maintainable by students.
- No real student personal data should be used during development and testing; only sample/mock data.

2.6 Assumptions and Dependencies

- It is assumed users have basic digital literacy and access to a browser and internet connection.
- It is assumed the platform is used within a single college/institution context (not cross-institution) for the current phase.
- The mock data layer is assumed to reset or persist only within the local browser session/storage, not across devices, until backend integration is completed.
- Actual video/voice conferencing depends on external third-party tools (e.g., Google Meet, Zoom) linked from within a session.

3. System Features (Functional Requirements)

This section describes each major feature of Campus Skill Hub along with its associated functional requirements, priority, and basic flow. Each feature maps directly to a use case described in Section 8.

1. User Registration & Authentication

Allows a new student to create an account and existing students to log in securely.

ID	Requirement	Priority
FR-01	The system shall allow a new user to register using name, college email, department, and password.	High
FR-02	The system shall validate that the email follows a valid college email format before registration.	High
FR-03	The system shall allow a registered user to log in using email and password.	High
FR-04	The system shall allow a user to log out of the platform.	Medium
FR-05	The system shall display appropriate error messages for invalid login/registration input.	Medium

Basic Flow

1. User navigates to the Register page and fills in required details.
2. System validates input fields (format, required fields, password strength).
3. On success, the account is created and the user is redirected to profile setup.
4. On subsequent visits, the user logs in via the Login page and is directed to the Dashboard.

3.2 Profile Management

Allows a user to build and maintain a profile describing themselves and their skills.

ID	Requirement	Priority
FR-06	The system shall allow a user to add/edit personal details (name, department, year, bio, profile photo).	High
FR-07	The system shall allow a user to add one or more skills they can teach, with a proficiency level (Beginner/Intermediate/Expert).	High
FR-08	The system shall allow a user to add one or more skills they want to learn.	High
FR-09	The system shall allow a user to set weekly availability (days/time slots) for teaching sessions.	Medium
FR-10	The system shall allow a user to delete or update previously listed skills.	Medium

Basic Flow

1. User opens 'My Profile' from the dashboard.

2. User adds/edits personal info and skill lists (teach/learn) via a form.
3. Changes are saved and reflected immediately on the user's public profile card.

3.3 Skill Discovery & Search

Allows users to find peers offering a desired skill or seeking a skill they can teach.

ID	Requirement	Priority
FR-11	The system shall display a browsable list of all available skills grouped by category.	High
FR-12	The system shall allow keyword search by skill name.	High
FR-13	The system shall allow filtering results by category, proficiency level, and department.	Medium
FR-14	The system shall display a peer's profile summary (name, skill, rating, availability) in search results.	High
FR-15	The system shall allow sorting search results by rating or relevance.	Low

Basic Flow

1. User enters a keyword or selects a category/filter on the Explore/Search page.
2. System returns a list of matching skill listings/peer profile cards.
3. User selects a listing to view the full peer profile.

3.4 Skill Exchange Requests

Allows a seeker to request a session with a provider, and the provider to respond.

ID	Requirement	Priority
FR-16	The system shall allow a seeker to send a skill-exchange request to a provider, including a proposed date/time and message.	High
FR-17	The system shall allow a provider to view incoming requests with seeker details.	High
FR-18	The system shall allow a provider to accept or decline a request.	High
FR-19	The system shall notify the seeker (in-app indicator) when a request status changes.	Medium
FR-20	The system shall allow either party to cancel a confirmed (not-yet-completed) session with a reason.	Medium

Basic Flow

1. Seeker views a provider's profile and clicks 'Request Session'.
2. Seeker fills a short request form (proposed time, message) and submits.
3. Provider receives the request under 'My Requests' and accepts or declines.
4. On acceptance, the session appears in both users' 'Upcoming Sessions' list.

3.5 Session Tracking & History

Allows users to track the status of their skill-exchange sessions over time.

ID	Requirement	Priority
FR-21	The system shall maintain a list of Upcoming, Completed, and Cancelled sessions per user.	High
FR-22	The system shall allow a user to mark a session as 'Completed' after the scheduled time.	Medium
FR-23	The system shall display session history with date, peer name, and skill exchanged.	Medium

Basic Flow

1. User opens 'My Sessions' to view session tabs (Upcoming/Completed/Cancelled).
2. After a session date passes, either participant can mark it Completed, unlocking the feedback form.

3.6 Ratings & Feedback

Builds trust in the platform by allowing peer feedback after each completed session.

ID	Requirement	Priority
FR-24	The system shall allow a user to rate their peer (1–5 stars) after a session is marked Completed.	High
FR-25	The system shall allow a user to leave an optional written review/comment.	Medium
FR-26	The system shall display the average rating on a user's public profile.	High
FR-27	The system shall prevent a user from rating the same completed session more than once.	Medium

Basic Flow

1. After marking a session Completed, the rating form is presented.
2. User submits star rating and optional comment.
3. The peer's average rating updates and the review is visible on their profile.

3.7 Notifications

Keeps users informed of important account activity within the app.

ID	Requirement	Priority
FR-28	The system shall show a notification badge/count for unread updates (new request, accepted/declined, new rating).	Medium
FR-29	The system shall display a chronological notification list accessible from the dashboard.	Low
FR-30	The system shall allow a user to mark notifications as read.	Low

Basic Flow

1. A triggering event occurs (e.g., request accepted).
2. A notification entry is created and the badge count increments.
3. User opens the notification panel; items are marked read on view.

3.8 Admin & Moderation

Provides basic administrative oversight of the platform's users and content.

ID	Requirement	Priority
FR-31	The system shall provide an Admin login separate from student accounts.	Medium
FR-32	The system shall allow the admin to view, search, and deactivate user accounts.	Medium
FR-33	The system shall allow the admin to add, edit, or remove skill categories.	Low
FR-34	The system shall allow the admin to view and act on reported users/reviews.	Medium
FR-35	The system shall display basic platform statistics to the admin (total users, active listings, sessions completed).	Low

Basic Flow

1. Admin logs in via a distinct Admin Dashboard route.
2. Admin reviews reported content/users and takes moderation action (warn/deactivate).
3. Admin manages the master list of skill categories used across the platform.

4. Data Requirements / Conceptual Data Model

This section defines the key data entities used by the system in the current mock-data phase and the future database phase. It is intended to guide both the frontend data structures (e.g., JSON objects) and the future relational schema.

1. Key Entities

Entity	Key Attributes
User	userId, name, email, passwordHash, department, year, bio, profilePhoto, role (student/admin), averageRating
Skill	skillId, name, category, description
UserSkillTeach	id, userId, skillId, proficiencyLevel
UserSkillLearn	id, userId, skillId, desiredLevel
Availability	id, userId, dayOfWeek, startTime, endTime
ExchangeRequest	requestId, seekerId, providerId, skillId, proposedDateTime, message, status (Pending/Accepted/Declined/Cancelled)
Session	sessionId, requestId, scheduledDateTime, status (Upcoming/Completed/Cancelled)
Rating	ratingId, sessionId, raterId, ratedUserId, stars, comment, createdAt
Notification	notificationId, userId, type, message, isRead, createdAt
SkillCategory	categoryId, name, description

4.2 Entity Relationship Summary

- A User can teach many Skills and wants to learn many Skills (many-to-many, via UserSkillTeach / UserSkillLearn).
- A User can have multiple Availability slots (one-to-many).
- An ExchangeRequest links exactly one Seeker (User) and one Provider (User) for one Skill.
- An accepted ExchangeRequest results in exactly one Session.
- A completed Session can generate up to two Ratings (one from each participant, rating the other).
- A SkillCategory groups multiple Skills (one-to-many).

5. External Interface Requirements

1. User Interfaces

The UI shall follow a consistent, professional visual theme suitable for a modern student-facing web platform, as summarized below. Screens are designed to be clean, minimal, and mobile-responsive.

UI Element	Color / Style
Titles / Cover Sections	Navy Blue
Main Headings	Royal Blue
Subheadings	Navy Blue
Tables / Section Highlights	Light Blue
Normal Body Text	Dark Gray
Page Background	White

Key Screens

Screen	Purpose
Landing Page	Introduces the platform and prompts Login/Register.
Register / Login	Account creation and authentication.
Dashboard	Overview of profile summary, notifications, and quick actions.
My Profile	View/edit personal info, teach/learn skills, availability.
Explore / Search	Browse and filter skills/peers.
Peer Profile View	View another student's public profile and 'Request Session' action.
My Requests	Manage incoming/outgoing exchange requests.
My Sessions	Upcoming, Completed, Cancelled sessions with rating access.
Notifications	Chronological list of platform notifications.
Admin Dashboard	Admin-only view for user, category, and content moderation.

5.2 Hardware Interfaces

The system has no direct hardware interface requirements. It runs entirely within a standard web browser on any device capable of rendering modern HTML5/CSS3/JavaScript.

5.3 Software Interfaces

Interface	Description
Web Browser	Chrome, Firefox, Edge, Safari (latest two versions) as the runtime environment.
Mock Data Layer (current phase)	Local JSON files / browser Local Storage simulating persistence.
Future REST API (Section 9)	JSON-based REST endpoints for authentication, profiles, requests, sessions, and ratings.
Future Database (Section 9)	Relational (e.g., MySQL/PostgreSQL) or NoSQL (e.g., MongoDB) data store.

5.4 Communication Interfaces

- Current phase: No external network communication required beyond serving the static frontend.
- Future phase: HTTPS-based REST API calls between frontend and backend server.
- Optional future integration: Email/SMTP service for notification emails; linking to third-party video-call URLs (Google Meet/Zoom) within a session.

6. Non-Functional Requirements

1. Performance

ID	Requirement
NFR-01	Core pages (Dashboard, Explore, Profile) shall load within 2 seconds under normal conditions with mock data.
NFR-02	Search and filter results shall be displayed within 1 second for the current dataset size.

6.2 Usability

ID	Requirement
NFR-03	The interface shall follow a consistent color theme and typography across all screens (see Section 5.1).
NFR-04	A first-time user shall be able to complete registration and profile setup within 5 minutes without external help.
NFR-05	The system shall provide clear, human-readable error and confirmation messages for all key actions.

6.3 Reliability

ID	Requirement
NFR-06	The system shall not lose locally stored user data (profile, requests) during normal browser sessions.
NFR-07	The system shall gracefully handle invalid or missing input without crashing the UI.

6.4 Availability

ID	Requirement
NFR-08	As a static/client-side application in the current phase, the platform shall be available whenever the hosting page is reachable (target 99% during evaluation period).

6.5 Security

ID	Requirement
NFR-09	Passwords shall not be stored or displayed in plain text in the UI (hashed/obscured even in the mock layer).
NFR-10	Admin routes/screens shall not be accessible to standard student accounts.
NFR-11	In the future backend phase, all API traffic shall use HTTPS and token-based authentication (e.g., JWT).

6.6 Maintainability

ID	Requirement
NFR-12	The codebase shall follow a modular component structure to allow easy replacement of the mock data layer with real APIs.
NFR-13	Naming conventions and folder structure shall be documented for future contributors.

6.7 Portability

ID	Requirement
NFR-14	The UI shall be responsive and function correctly on screen widths from 360px (mobile) to 1920px (desktop).

6.8 Accessibility

ID	Requirement
NFR-15	The UI shall maintain sufficient color contrast between the Dark Gray text and White background per WCAG 2.1 AA guidelines.
NFR-16	All interactive elements (buttons, forms) shall be keyboard-navigable.

7. System Architecture Overview

The current phase follows a simple, layered frontend architecture designed to evolve smoothly into a full client-server architecture in future phases.

1. Current Phase Architecture (Frontend-Only)

- **Presentation Layer:** React (or HTML/CSS/JS) components for each screen listed in Section 5.1.
- **State/Logic Layer:** Application state management (e.g., React Context/Redux or plain JS state) handling user session, requests, and navigation.
- **Mock Data Layer:** JSON-based sample data and browser Local Storage simulating a database and API responses.

7.2 Future Phase Architecture (Full Stack)

- **Client:** Same presentation layer, now calling real REST APIs instead of mock data.
- **Application/API Layer:** Node.js/Express (or equivalent) exposing REST endpoints for auth, profiles, requests, sessions, ratings.
- **Data Layer:** Relational/NoSQL database implementing the entities in Section 4.
- **Hosting:** Frontend and backend deployed on a cloud platform (e.g., Vercel/Netlify for frontend, Render/Railway for backend).

7.3 High-Level Component Diagram (Textual)

Layer	Components
Presentation	Landing, Auth, Dashboard, Profile, Explore, Requests, Sessions, Notifications, Admin
Logic / State	Session Manager, Request Handler, Notification Manager, Rating Manager
Data (Current)	Mock JSON Data Store / Local Storage
Data (Future)	REST API Client -> Backend API -> Database

8. Use Case Diagrams and Descriptions

The table below summarizes the primary use cases of the system. Actors are the Student (acting as Seeker and/or Provider) and the Administrator. A simplified textual use-case diagram description follows, suitable for redrawing as a UML use case diagram during design.

Use Case	Primary Actor	Description
Register / Login	Student	Create an account or authenticate into the platform.
Manage Profile	Student	Add/update personal info and teach/learn skill lists.
Search Skills/Peers	Student	Browse and filter available skills and peer profiles.
Send Exchange Request	Student (Seeker)	Request a learning session from a provider.
Respond to Request	Student (Provider)	Accept or decline an incoming request.
Manage Sessions	Student	Track and update the status of scheduled sessions.
Rate & Review Peer	Student	Submit feedback after a completed session.
View Notifications	Student	View updates related to requests and ratings.
Moderate Platform	Administrator	Manage users, categories, and reported content.

8.1 Sample Detailed Use Case: Send Exchange Request

Field	Description
Use Case ID	UC-04
Actor	Student (Seeker)
Precondition	Seeker is logged in and viewing a Provider's profile.
Main Flow	1) Seeker clicks 'Request Session'. 2) Fills proposed date/time and message. 3) Submits request. 4) System creates request with status 'Pending' and notifies the Provider.
Alternate Flow	If required fields are missing, the system displays a validation error and does not submit.
Postcondition	A new ExchangeRequest record exists with status 'Pending', visible to both users.

9. Future Scope / Planned API and Database Integration

This section outlines how the current frontend-only system is designed to evolve into a full-stack application, to guide future development phases and to demonstrate forward-looking system design during evaluation.

1. Planned REST API Endpoints (Sample)

Method & Endpoint	Purpose
POST /api/auth/register	Register a new student user.
POST /api/auth/login	Authenticate a user and issue a session token.
GET /api/users/{id}	Fetch a user's public profile.
PUT /api/users/{id}	Update profile details and skill lists.
GET /api/skills?query=&category;=	Search/filter skills and associated peers.
POST /api/requests	Create a new skill-exchange request.
PUT /api/requests/{id}	Accept, decline, or cancel a request.
GET /api/sessions?userId=	Retrieve a user's sessions (upcoming/completed/cancelled).
POST /api/ratings	Submit a rating/review for a completed session.
GET /api/notifications?userId=	Retrieve notifications for a user.

9.2 Planned Enhancements

- Migrate mock data layer to a real backend with persistent database (see Section 4 for schema).
- Add email notifications for key events (request received, accepted).
- Introduce a basic skill-matching recommendation feature based on shared interests/department.
- Add in-app chat between matched peers prior to session confirmation.
- Add calendar integration (e.g., Google Calendar) for scheduled sessions.

10. Testing Requirements

This section defines the testing approach to validate that the implemented system meets the requirements defined in this document.

1. Testing Levels

Level	Focus	Example
Unit Testing	Individual components/functions	Validate email format function; skill filter logic
Integration Testing	Interaction between components/modules	Sending a request updates both Seeker's and Provider's request lists
System Testing	End-to-end functional flow	Full flow: Register -> Set up profile -> Search -> Request -> Accept -> Complete -> Rate
UI/Usability Testing	Interface consistency and ease of use	Verify color theme, responsiveness, and navigation clarity
Acceptance Testing	Validation against this SRS by faculty/evaluators	Demo walkthrough mapped to Section 3 functional requirements

10.2 Sample Test Cases

Test ID	Requirement	Test Description	Expected Result
TC-01	FR-01/FR-02	Register with an invalid email format	System shows validation error; account not created
TC-02	FR-16	Seeker sends a request with all fields filled	Request created with status 'Pending'
TC-03	FR-18	Provider declines a pending request	Request status changes to 'Declined'; seeker notified
TC-04	FR-24/FR-27	User attempts to rate the same session twice	System blocks the second rating attempt
TC-05	NFR-10	Student account attempts to access Admin Dashboard URL	Access denied / redirected

11. Other Requirements

- **Legal/Institutional:** The platform should restrict registration to institutional/college email addresses to keep the community verified.
- **Data Privacy:** Only necessary profile information should be collected; no sensitive personal data beyond academic details.
- **Branding:** The platform name 'Campus Skill Hub' and the defined color theme should be used consistently across all deliverables (UI, documentation, presentation).
- **Documentation:** This SRS should be maintained and updated as requirements evolve during development (version history in Document Control).

Appendix A: Glossary

Term	Meaning
Peer-to-Peer Learning	A learning model where students teach and learn directly from one another, without a formal instructor.
Proficiency Level	A self-declared indicator (Beginner/Intermediate/Expert) of a user's skill level in a given subject.
Mock Data Layer	Temporary simulated data used during the current development phase in place of a live backend/database.

Appendix B: Analysis Models

The following diagrams are recommended to be produced during the design phase, based on the models described in this SRS, and included in the final project report/presentation:

- UML Use Case Diagram — derived from Section 8.
- Entity Relationship (ER) Diagram — derived from Section 4.
- System Architecture Diagram — derived from Section 7.
- Wireframes/Mockups for each screen listed in Section 5.1, using the specified color theme.
- Data Flow Diagram (Level 0 and Level 1) illustrating the flow between Student, Provider, and the platform.

— End of Document —